



Software-Architekturen

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

Die Vorlesung basiert im Wesentlichen auf Literatur und Materialien aus folgenden Quellen.

- Gorton, Ian:
Essential Software Architecture, 2. ed. Springer, 2011.
- Clements, P.; Bass, L. and Kazman, R.:
Software Architecture in Practice, 2. ed. Addison-Wesley, 2003.
- Vogel, O.; et al.:
Software-Architektur: Grundlagen – Konzepte – Praxis, 2. Auflage. Spektrum, 2009.
- Avgeriou, P.; et al. (editors):
Relating Software Requirements and Architectures. Springer, 2011.
- Larman, Craig:
Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3. ed. Prentice Hall, 2004.
- Sommerville, Ian:
Software Engineering, 9. Auflage. Pearson Studium, 2012.
- Fowler, M.:
Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.
- Richardson, C.:
Microservices Patterns: With Examples in Java. Manning, 2019.

- Heineman, G. T. and Councill, W. T.:
Component-Based Software Engineering: Putting the Pieces Together. Addison-Wesley, 2001.
- Gamma, Erich et al.:
Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1995.
- Fowler, Martin: Analysemuster:
Wiederverwendbare Objektmodelle. Addison-Wesley, 1999.
- Pattern-Oriented Software Architecture, Volume 1:
Buschmann, Frank:
A System of Patterns. John Wiley & Sons, 1996.
- Pattern-Oriented Software Architecture, Volume 2:
Schmidt, Douglas C.:
Patterns for Concurrent and Networked Objects. John Wiley & Sons, 2000.
- Pattern-Oriented Software Architecture, Volume 3:
Kircher, M. and Prashant, J.:
Patterns for Resource Management. John Wiley & Sons, 2004.
- Pattern-Oriented Software Architecture, Volume 4:
Buschmann, Frank; Henney, Kevlin; and Schmidt, Douglas C.:
A Pattern Language for Distributed Computing. John Wiley & Sons, 2007.
- Vernon, V.:
Domain-Driven Design kompakt. dpunkt.verlag, 2017.

- Reussner, R. und Hasselbring, W. (Hrsg.):
Handbuch der Software-Architektur. dpunkt, 2006.
- Goll, J. und Dausmann, M.:
Architektur- und Entwurfsmuster der Softwaretechnik. Springer Vieweg, 2013.
- Horn, E. und Reinke, Th.:
Softwarearchitektur und Softwarebauelemente. Hanser, 2002.
- Andresen, A.:
Komponentenbasierte Softwareentwicklung mit MDA und UML 2 und XML, 2. Auflage. Hanser, 2004.
- Rupp, C., Queins, S. und Zengler, B.:
UML 2 glasklar: Praxiswissen für die UML-Modellierung, 3. Auflage. Hanser, 2007.
- OMG Object Management Group:
Unified Modeling Language: Infrastructure / Superstructure (2.4.1). OMG, 2011.
- OMG Object Management Group:
Unified Modeling Language (2.5). OMG, 2013.



Software-Architekturen

—

Was ist Architektur?

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe

University of
Applied Sciences

Fakultät für

**Informatik und
Wirtschaftsinformatik**

Architektur ist das zentrale Glied in dieser Systematik des Vorgehens.

+IKA Was ist Architektur?



- **Salisbury Cathedral**
„one of the leading examples of Early English architecture” [wiki]
- **Stonehenge**
„the most architecturally sophisticated prehistoric stone circle in the world” [UNESCO. World Heritage List, 373. whc.unesco.org]

+IKA Was ist Architektur?

„[...], but only perfect proportions and colors in a well-balanced harmony deserve that title of honor: beautiful. Just the same is true in architecture. **Only perfect harmony in its technical functions as well as in its proportions can result in beauty.** That makes our task so manifold and complex.

**[Walter Gropius: Architecture at Harvard University.
In Architectural Record, 1937.]
(online – archrecord.construction.com)**

[IEEE-SA Standards Board. IEEE Std 1471-2000 – IEEE Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE Computer Society, 2000.]

Eine systematische Betrachtung:

- **Architecture:**
The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.
- **Architecting:**
The activities of defining, documenting, maintaining, improving, and certifying proper implementation of an architecture.
- **Architect:**
The person, team, or organization responsible for systems architecture.
- **Architectural Description (AD):**
A collection of products to document an architecture.

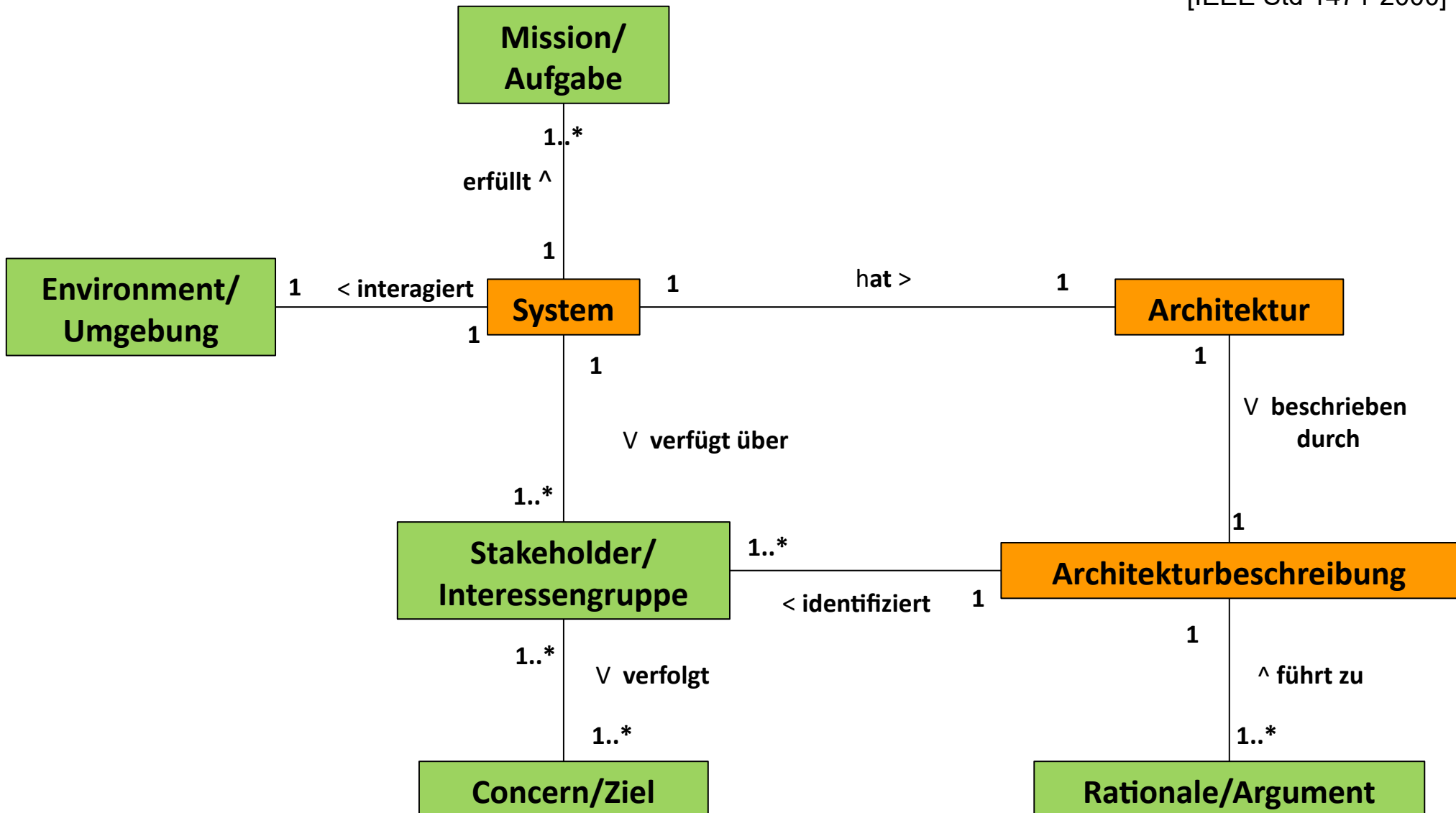
+IKA Was ist Architektur?

[IEEE-SA Standards Board. IEEE Std 1471-2000 – IEEE Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE Computer Society, 2000.]

- **System:**
A collection of components organized to accomplish a specific function or set of functions.
- **System Stakeholder:**
An individual, team, or organization (or classes thereof) with interests in, or concerns relative to, a system.
 - Designer,
 - Entwickler,
 - Projektmanager,
 - Produktmanager,
 - Vertriebsmanager,
 - Kunde, ...



Architekt



+IKA Was ist Architektur?

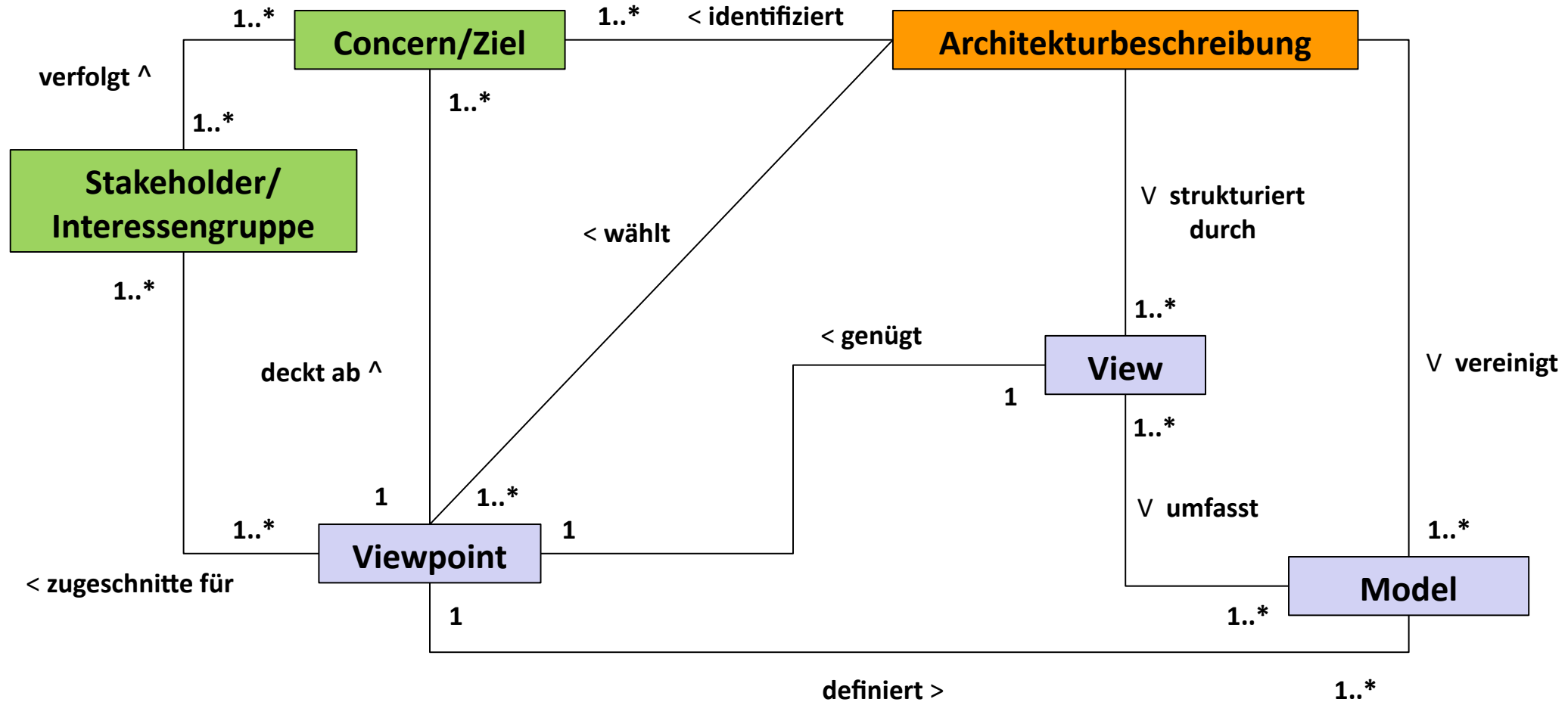
[IEEE-SA Standards Board. IEEE Std 1471-2000 – IEEE Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE Computer Society, 2000.]

- **View:**

A representation of a whole system from the perspective of a related set of concerns.

- **Viewpoint:**

A specification of the conventions for constructing and using a view. A pattern or template from which to develop individual views by establishing the purposes and audience for a view and the techniques for its creation and analysis.



Concerns sind vielfältig und abhängig von der jeweiligen Interessensgruppe, aber elementar für die entstehende Architektur.

Concerns sind:

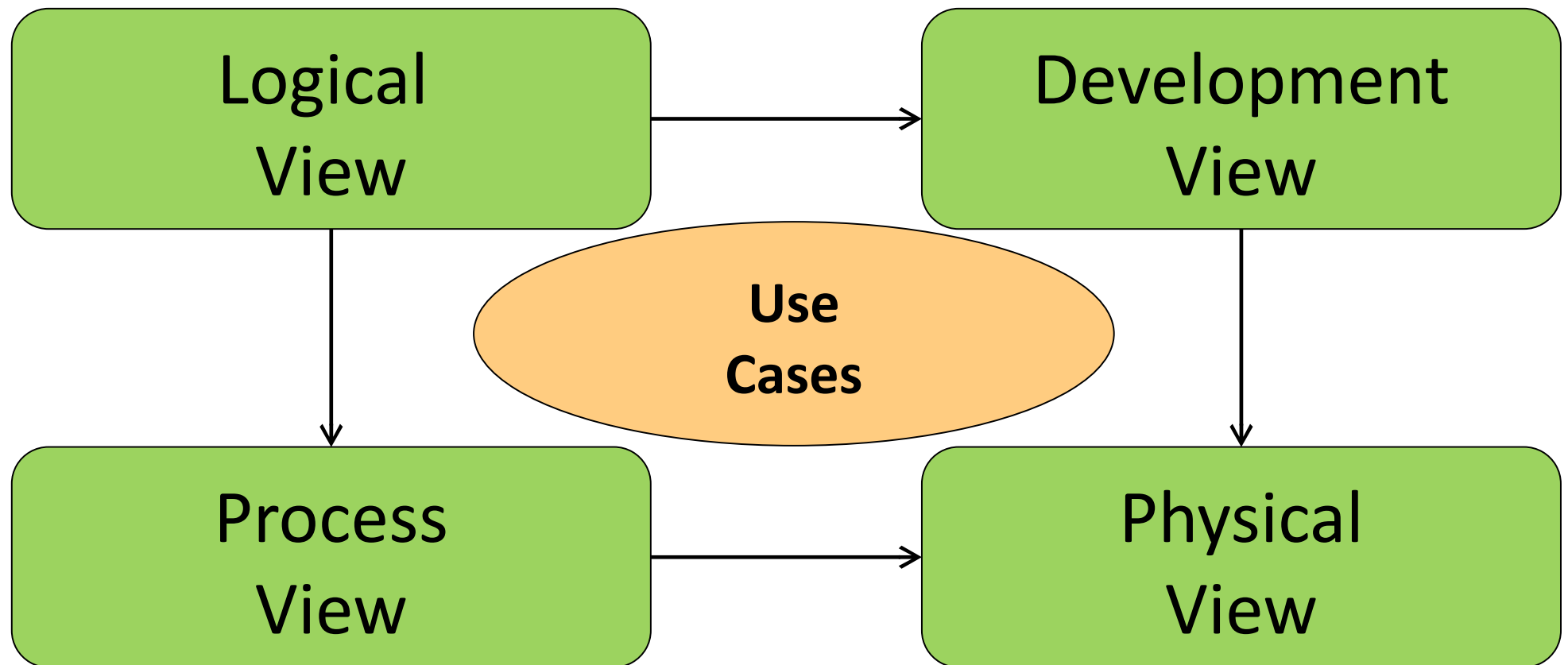
- Zentrale Anforderungen,
- Unabdingbare Constraints,
- Lediglich Absichten,
- Vage Ideen



Oftmals sind diese Dinge, die ein Stakeholder bewegen, nur schwer zu entdecken und doch entscheidend für den Erfolg des Projekts.

+HKA 4+1 View Model

[Philippe Kruchten: Architectural Blueprints – The “4+1” View Model of Software Architecture. In IEEE Software 12 (6),1995.]



Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Logical View :**
Sie beschreibt Strukturen der Architektur aus Anwendersicht.

Modelliert werden die Zusammenhänge zwischen den zentralen Bausteinen. Wie diese **strukturiert** und organisiert sind. Wie sie sich aus kleineren zusammensetzen, wie sie miteinander verbunden sind und **zusammenarbeiten**, um die funktionalen Anforderungen, die an das System gestellt werden, zu erfüllen.

Class Diagrams, Sequence Diagrams, Component Diagrams, usw.

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Process View:**
Sie beschreibt die Abläufe innerhalb der Architektur.

Im Fokus stehen Nebenläufigkeit und die Verteilung der Aufgaben auf selbständig agierende Entitäten (vom einzelnen Thread bis zum komplexen Server) und deren **Kommunikation und Synchronisation**. Adressiert werden dabei nicht-funktionale Anforderungen wie Performance, Verfügbarkeit und Fehlertoleranz.

Activity Diagrams, Interaction Overview Diagrams, Sequence Diagrams, Composite Structure Diagrams, usw.

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Physical View:**
Sie beschreibt die reale (physische) Struktur.

Im Fokus steht die Verteilung der realen „Baugruppen“ und deren Aufgaben auf die realen Hardware-Komponenten (Knoten).
Adressiert werden insbesondere nicht-funktionale Anforderungen wie Skalierbarkeit, Ausfallsicherheit und Performance.

Deployment Diagrams

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Development View:**
Sie beschreibt die Verteilung der zu entwickelnden Software-Elemente (Pakete/Module/Bibliotheken) aus Entwicklersicht.

Im Vordergrund steht die Strukturierung des Systems aus Sicht der Entwickler und dessen Konfigurationsmanagement. Welche Bibliothek stellt welche Schnittstelle bereit? Wie feingranular sind die einzelnen Bibliotheken? Welches Paket beinhaltet welche Unterpakete, usw.?

Deployment Diagrams, Package Diagrams, Class Diagrams, usw.

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Use Case View:**
Sie beschreibt die funktionalen Anforderungen an das System mittels Use Cases und Szenarien.

Im Vordergrund steht die Beschreibung von Abläufen, die besonders viel Information zur Architektur des Systems beitragen. Aufgrund der integralen Sichtweise mit dem Fokus auf die wichtigsten Bedürfnisse der Anwender sind diese Use Cases das Maß, an dem sich die Architektur orientieren muss.

Use Diagrams, Activity Diagrams, usw.

+IKA Ist dies noch modern?

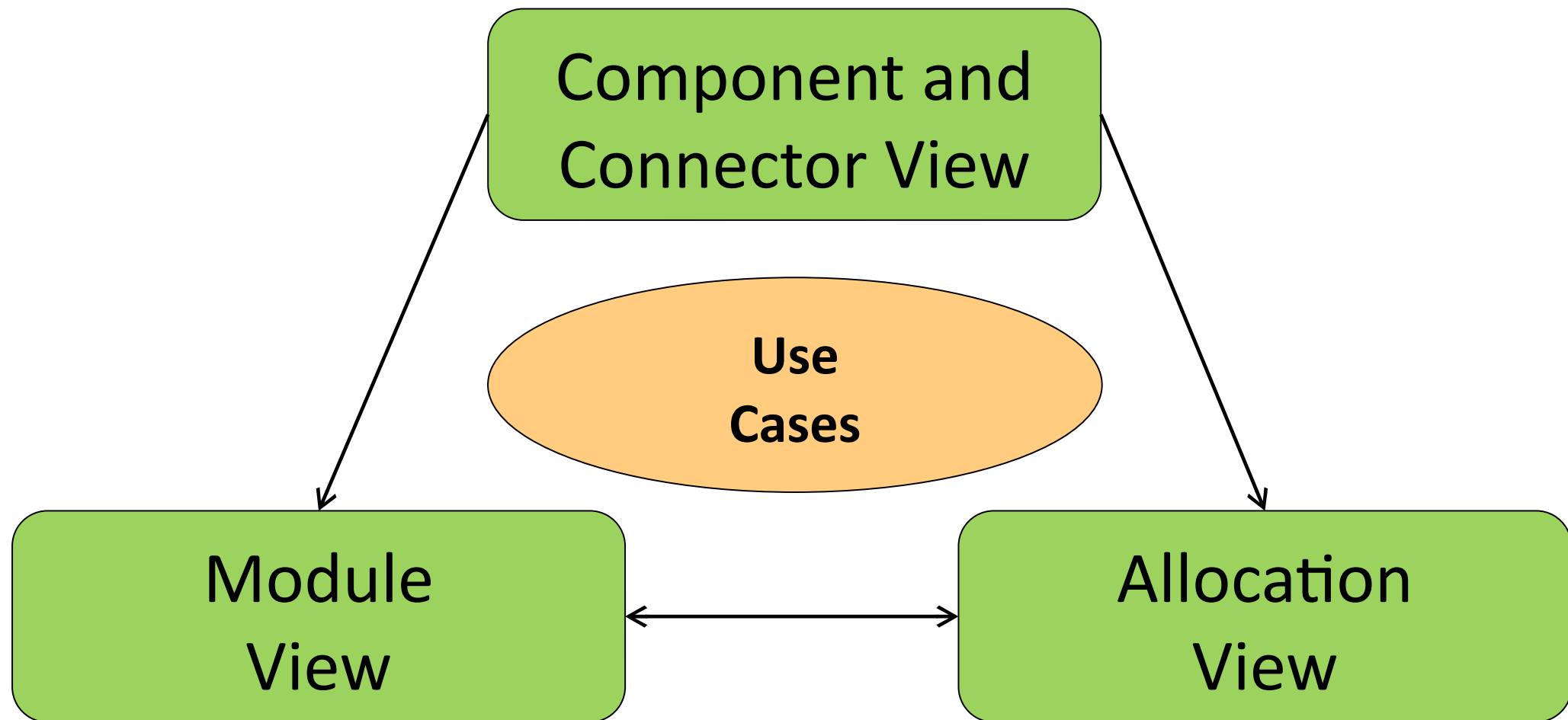
Oftmals ist eine Abgrenzung der Betrachtung der unterschiedlichen Modelle (Viewpoints) und der damit verbundenen Zuordnung zu speziellen Stakeholder-Gruppen nur bedingt möglich.

**Insbesondere stellt sich die Frage:
Wie trennt man logische Sicht, Entwicklersicht und
Prozesssicht?**

+HKA 3+1 View Model

Clements, P. et. al., editors. Documenting Software Architectures: Views and Beyond (2nd Edition). Addison-Wesley, 2010

Christensen, H.B; Corry, A. and Hansen, K.M. An Approach to Software Architecture Description Using UML: Revision 2.0



Component and Connector View

- **Beschrieben wird, was das System „macht“, d. h. die dynamischen Abläufe (Funktionen, Daten- und Kontrollfluss).**

Komponenten:

Objekte, Prozesse, Threads, usw. Sie repräsentieren die für die Erbringung der Funktionalität verantwortlichen Teile des Systems.

Konnektoren:

Shared Memory, Sockets, JDBC, ESB, usw. Sie beschreiben, wie die Komponenten miteinander verbunden sind, auf welche Art und Weise Kontrollfluss und Daten von einer Komponente zur anderen übergehen.

- **Stakeholder:**
Architekten, Entwickler, Kunden, Anwender

Module View

- Beschrieben werden Strukturen, Klassen, Interfaces, die Pakete, denen sie zugeordnet sind und die Beziehungen untereinander...

Im Zentrum der Beschreibung steht die Umsetzung der Funktionalität, deren Zerlegung und Verteilung.

Die Modularisierung der Software

- Stakeholder:
Architekten, Entwickler

Allocation View

- **Beschrieben werden Bibliotheken, deren Konfigurationsmanagement, Deployment bis hin zur Zuordnung auf die einzelnen Entwicklungsteams.**
 - **Deployment:**
Zuordnung von Software-Komponenten zu Hardware.
 - **Implementierung:**
Zuordnung von Software-Komponenten zu Dateistrukturen (im Rahmen der Entwicklung, Konfiguration und Integration).
 - **Arbeitspakete:**
Zuordnung von Software-Komponenten zu Entwicklungsteams.

Allocation View

Im Zentrum der Beschreibung steht die Software und ihre Beziehung zur realen Entwicklungs- und Ausführungsumgebung.

- **Stakeholder:**
Architekten, Entwickler, Betreiber, Kunden, Anwender, ...

Alle

+IKA Was ist Architektur? Essentials

Die Architektur eines Softwaresystems ist ein Bauplan für die Organisation des Gesamtsystems vom Entwurf über die Entwicklung bis zum Betrieb.

Die Aufgabe der Architektur ist es, die signifikanten (hervorstechenden) statischen und dynamischen Aspekte des Gesamtsystems zu erfassen und zu organisieren, d. h. die Struktur der Subsysteme und Komponenten, deren Interfaces, Verbindungen, Interaktion, Verantwortlichkeiten und wie sie zusammenarbeiten.

Hierzu gehören Aspekte wie:

- Verwendung,
- Leistung,
- Technologie,
- Realisierung,
- Konfiguration, ...

**Auch die Beschreibung der Architektur ist strukturiert.
Unterschiedliche Views adressieren unterschiedliche
Gesichtspunkte:**

Zentrale Gesichtspunkte sind:

- Modularisierung und Struktur,
- Modularisierung und Kommunikation,
- Modularisierung und Deployment (Bereitstellung),
Konfiguration, Integration und Arbeitsteilung